

Model Driven Design



Domain-Driven Design

DDD is an approach to developing software that deeply connects the **implementation** to the core **business concepts**

it connects the code to the domain,
to the context of our application

but how can we encapsulate the domain inside our code?



The Domain Model

the best way to encapsulate the domain is to make a **software-reflection** of it

the software incorporates the core concepts and elements of the domain and draws the **relationships** between them

in other terms, the software must **model** the domain



What Is a Domain Model?

according to **Eric Evans**, a domain model is not a particular diagram; it is the **idea** that the diagram is intended to convey

it is a rigorously organized and selective
abstraction of domain knowledge

the model is the **internal representation** of the target domain



Code Design vs Software Design

software design is like creating the architecture of a house – it's about the **big picture**

code design is working on the details, like the location of a painting on a certain wall

when we have a model expressed,
we can start doing code design



Design & Development

DDD combines **software design** and **development practices** to create a better solution

good design will **accelerate** development,
while feedback from development will **enhance** the design



The Communication Barrier

developing a model requires **developers** and **domain experts** to work together

but there is a fundamental **communication barrier** between them

developers think in classes, methods, and patterns
domain experts know nothing about software



The Ubiquitous Language

a core principle of DDD is to create a **common language** between developers and domain experts

since the **model** is the common ground, it should be used as the building foundation for this language

the team must use this language **consistently** in all communications, and in the code as well



Building the Language

this language does not appear overnight
it takes **hard work** and focus

it must appear consistently in **speech**,
writings, diagrams, and **code**

it should be fully exercised to discover
key domain concepts for the model



The Modeling Process

any domain can be expressed with **many models**, and
any model can be expressed in various ways in code

for each problem there can be more than one solution
so how do we choose the **best model**, and how do we
approach the transition from **model to code**?



The Analysis Model

one technique is the **analysis model**, where the model results from business domain analysis only, with **no consideration** for software

analysts can't foresee defects in their model, leaving important details to be discovered during implementation

a truthful model could have serious problems with **persistence** or **performance**



A Better Approach

closely relate domain **modeling** and **design** – construct the model with an eye open to software considerations

developers should be included in the modeling process

tightly relating code to the model gives the code **meaning** and makes the model **relevant**



Object-Oriented Programming

tying the implementation to a model requires tools that support a **modeling paradigm**

OOP is suitable because both OOP and domain modeling are based on the same paradigm

classes, associations, object instances, and messaging create direct mappings between model and code



The Building Blocks

creating the model alongside domain experts using a
ubiquitous language leads to **model-driven design**

now let's look at the main **building
blocks** of a model-driven design

the key elements of **object modeling**
and **software design** from the DDD viewpoint



Layered Architecture

a large part of any application is not directly related to the **domain** but serves the infrastructure

we need a **layered design** where each layer is cohesive and depends only on those below it

all code related to the domain model must be **isolated** in one layer, separate from UI, application, and infrastructure



The Four Layers

User Interface: presents information and interprets user commands

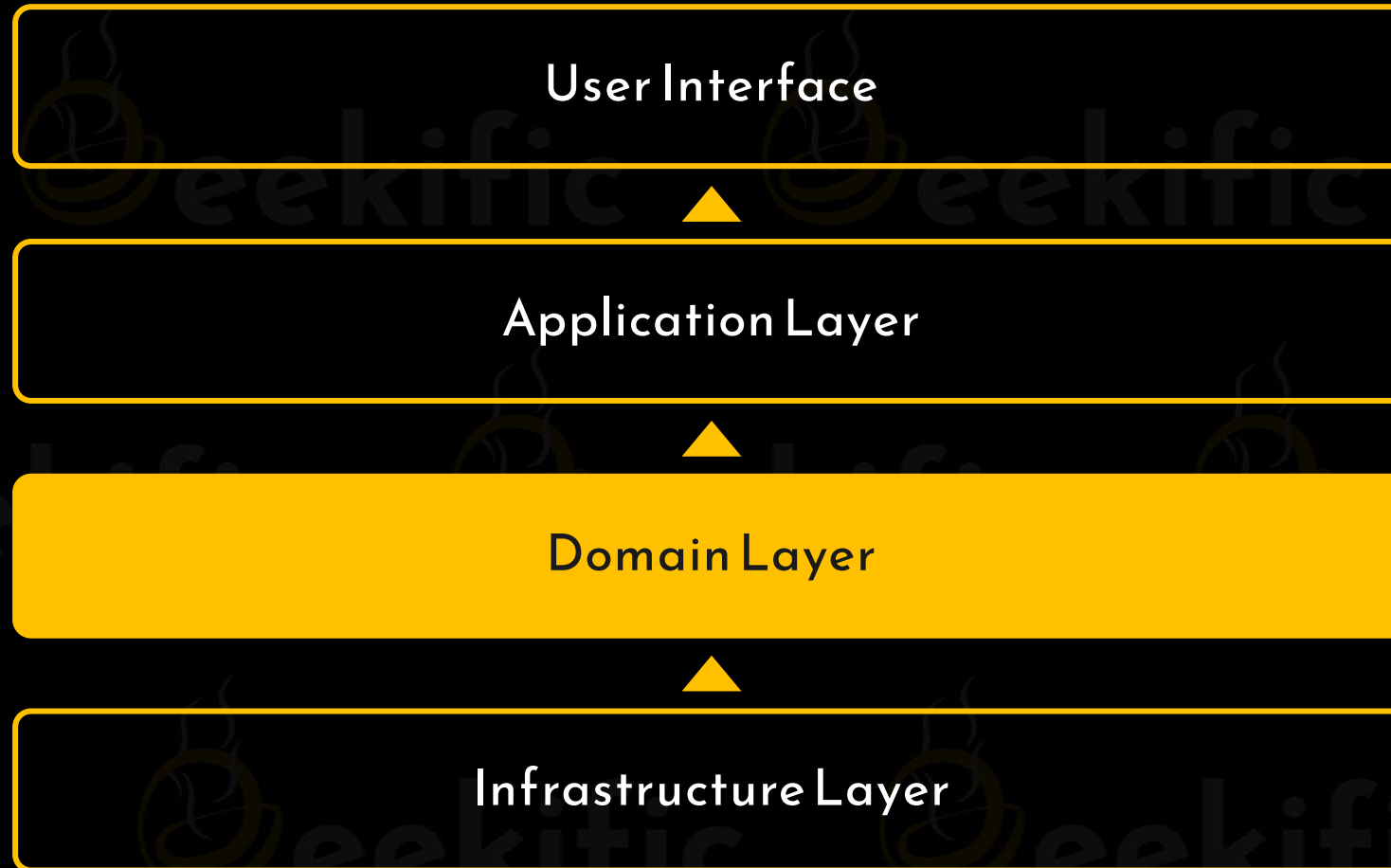
Application Layer: coordinates activity, holds task progress, no business logic

Domain Layer: the heart of the business software, holds business object state

Infrastructure Layer: supports all layers with communication and persistence



Layered Architecture



Entities

some objects have an **identity** that remains the same throughout all states of the software

for these objects, it is not the **attributes** that matter but a thread of **continuity** and identity

implementing entities means creating identity – keep the class definition **simple** and focused on lifecycle continuity



Value Objects

consider a Point in a drawing application – it has **coordinates** but does it need an identity?

we are not interested in **which** object it is,
but **what** attributes it has

objects that describe aspects of a domain
with **no identity** are named Value Objects



Value Object Properties

not all objects should be entities – distinguish between **Entity Objects** and **Value Objects**

having no identity means Value Objects can be easily **created** and **discarded**

value objects should be **immutable** – created with a constructor and never modified during their lifetime



Services

some domain aspects are not easily mapped to objects
– e.g., **transferring money** between accounts

a **Service** provides domain functionality without internal state

three characteristics: refers to a **domain concept**,
references **other objects**, and is **stateless**



Modules

for large models, it is necessary to organize into **modules** to reduce complexity

group highly related classes to provide maximum **cohesion** and minimum **coupling**

modules should have **well-defined interfaces** accessed by other modules to reduce connections



Object Lifecycle

domain objects go through states:
created, placed in memory, used, and **destroyed**

managing the lifecycle is a challenge that
can negatively impact the domain model

Aggregates define ownership and boundaries,
Factories handle creation, **Repositories** handle storage



Aggregates

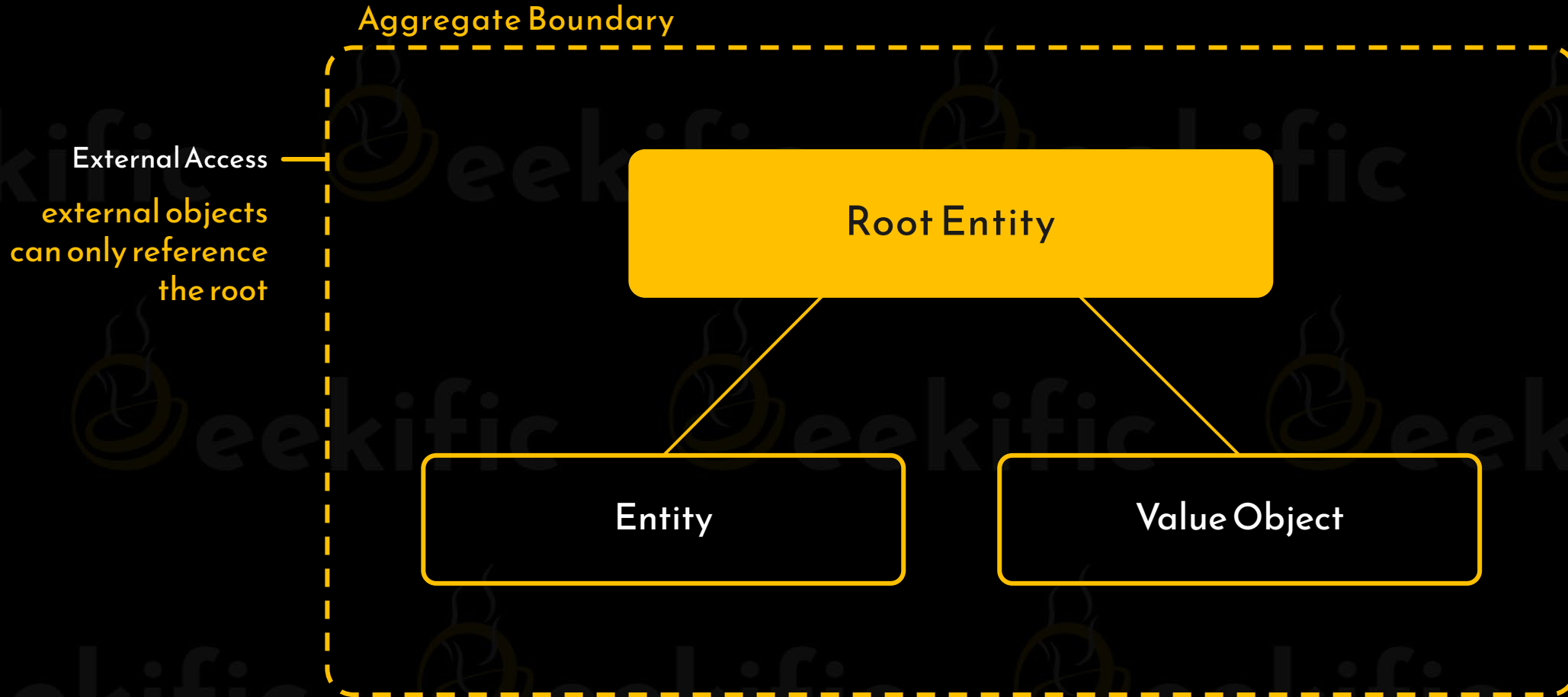
a group of associated objects considered as **one unit** regarding data changes

each aggregate has one **root** – an Entity that is the only object accessible from outside

external objects can hold references **only to the root**, ensuring data integrity and enforcing invariants



Aggregate Structure



Aggregate Rules

other objects cannot directly change objects inside – they can only change the **root** or ask it to perform actions

cluster **Entities** and **Value Objects** into Aggregates and define boundaries around each

only the root should be obtainable through **queries**, other objects through traversal associations



Factories

Entities and Aggregates can be too complex to create in a constructor

Factories encapsulate the knowledge necessary for object creation, especially useful for **Aggregates**

when the root is created, all contained objects are created and all **invariants** are enforced – the process must be **atomic**



Repositories

having clients access the database directly swamps code and makes the **domain irrelevant**

Repositories encapsulate all logic needed to obtain object references – a storage place for **globally accessible** objects

the **Factory** creates new objects, the **Repository** finds already created ones



Factory vs Constructor

favor a simple **constructor** when construction is not complicated and does not involve creating other objects

use a **Factory** when creation involves other objects, enforcing invariants, or choosing implementations

when adding to a Repository, create with the Factory **first**, then give it to the Repository to store



Summary

DDD provides **building blocks** for model-driven design: Layered Architecture, Entities, Value Objects, Services, and Modules

Aggregates define object boundaries, **Factories** handle creation, and **Repositories** handle storage

together, these patterns keep the **domain model** at the center of the software design

